



MACHINE LEARNING CON PYTHON

TEMA 6. APRENDIZAJE
SUPERVISADO: REGRESIÓN Y
CLASIFICACIÓN



Contenidos Tema 6

- / 1. Aprendizaje Supervisado I: Regresión
- / 2. Aprendizaje Supervisado II: Clasificación
- / 3. Evaluación y validación de modelos de ML

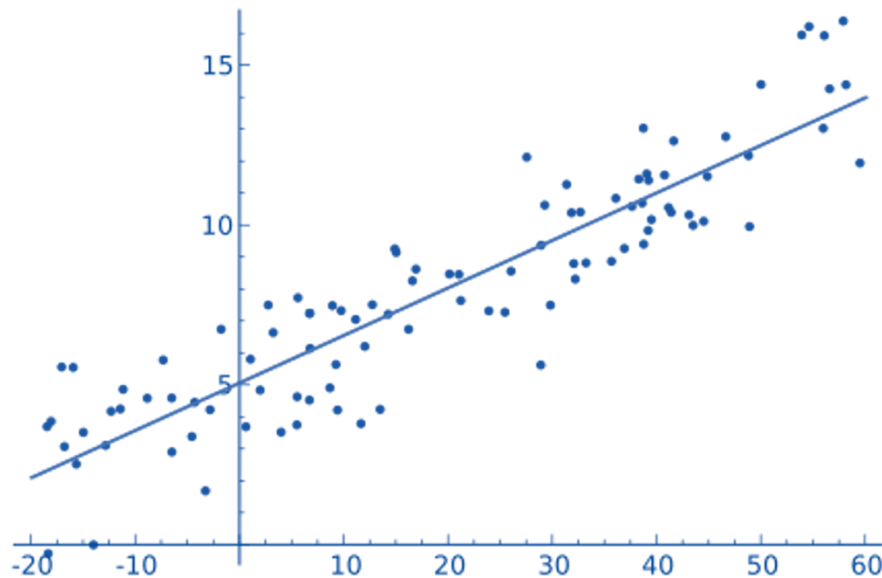
ENLACE A GITHUB CON LOS MATERIALES:

<https://github.com/IconoTC/Machine-Learning-con-PYTHON>

REPOSITORIO DE DATASETS PARA EL CURSO: <https://github.com/gakudo-ai/open-datasets>

CONSTRUCCIÓN Y ENTRENAMIENTO DE MODELOS DE REGRESIÓN

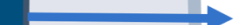
Regresión: aprender a predecir una variable numérica a partir de uno o varios atributos predictores



**House
attributes**



**Regression
Model**



Price



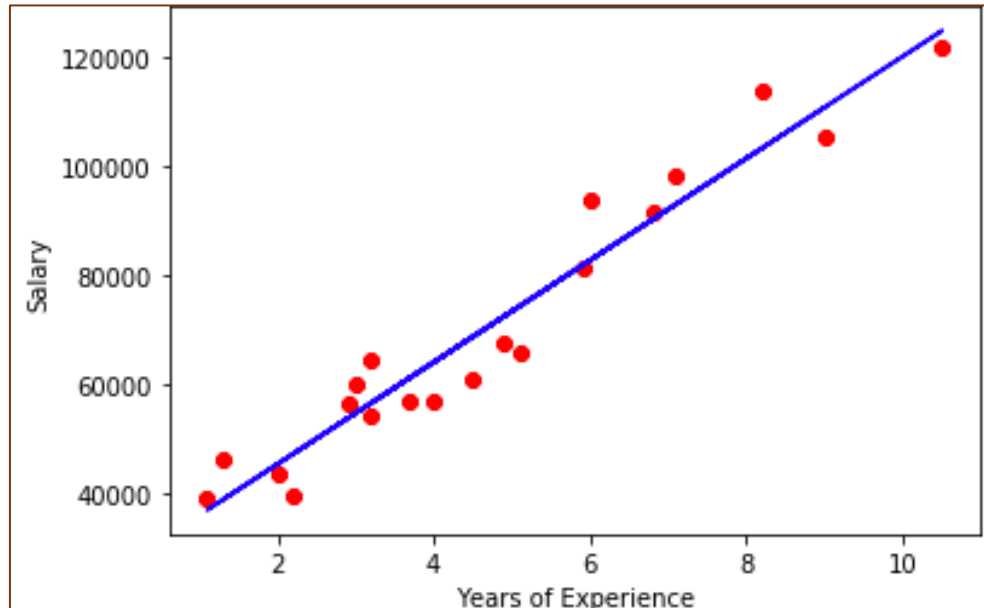
A dark blue icon of a price tag with a small circle at the top.

CONSTRUCCIÓN Y ENTRENAMIENTO DE MODELOS DE REGRESIÓN

Regresión lineal simple: Tal vez el modelo de ML supervisado más sencillo de todos

- Estimar variable numérica de salida y a partir de una variable de entrada x , asumiendo una relación lineal entre ambas.

$$\hat{y} = mx + n$$

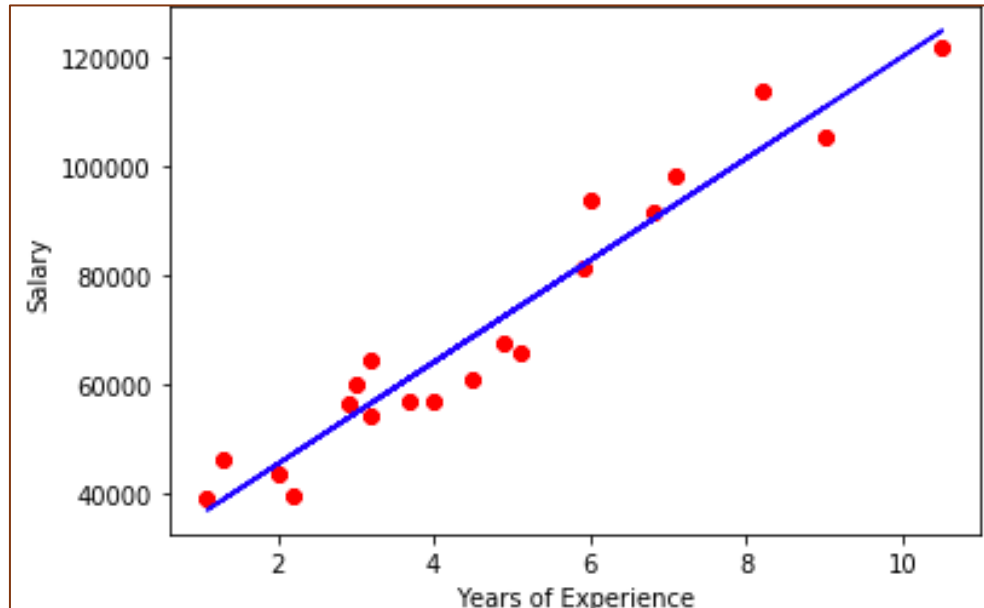


CONSTRUCCIÓN Y ENTRENAMIENTO DE MODELOS DE REGRESIÓN

Regresión lineal simple: Tal vez el modelo de ML supervisado más sencillo de todos

- Estimar variable numérica de salida y a partir de una variable de entrada x , asumiendo una relación lineal entre ambas.

$$\hat{y} = mx + n$$



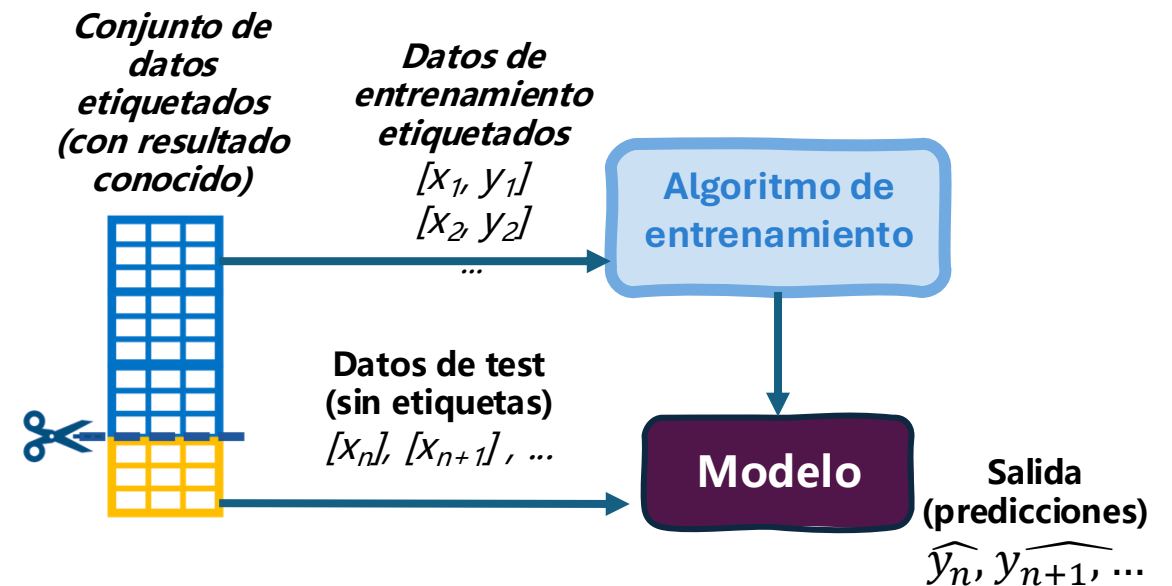
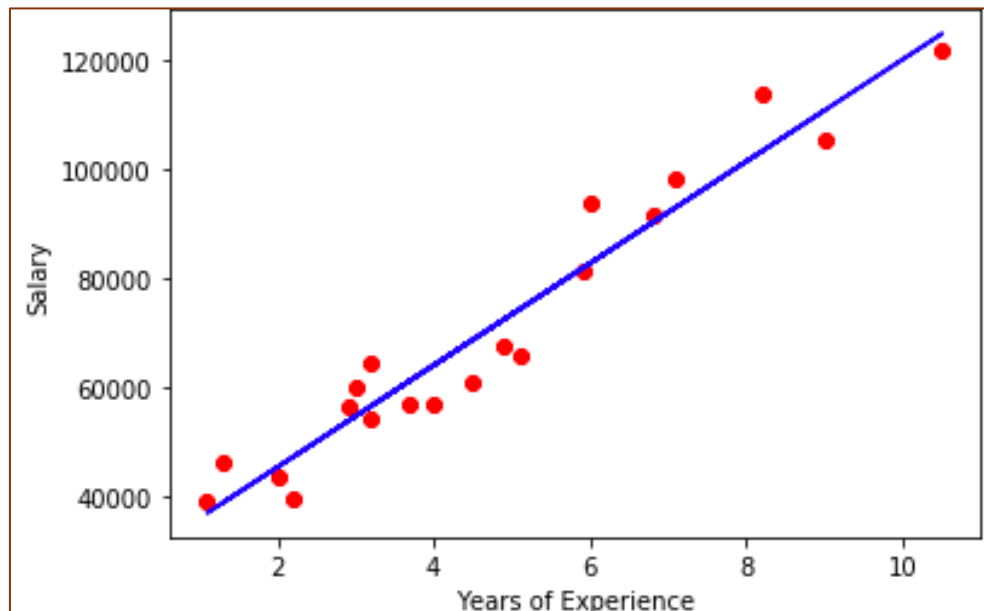
El modelo aprendido es la **función de una línea recta**, caracterizada por dos parámetros (m , n). Será mejor cuanto más fielmente se ajuste dicha línea a un conjunto de datos de entrenamiento (puntos)

CONSTRUCCIÓN Y ENTRENAMIENTO DE MODELOS DE REGRESIÓN

Regresión lineal simple: Tal vez el modelo de ML supervisado más sencillo de todos

- Estimar variable numérica de salida y a partir de una variable de entrada x , asumiendo una relación lineal entre ambas.

$$\hat{y} = mx + n$$

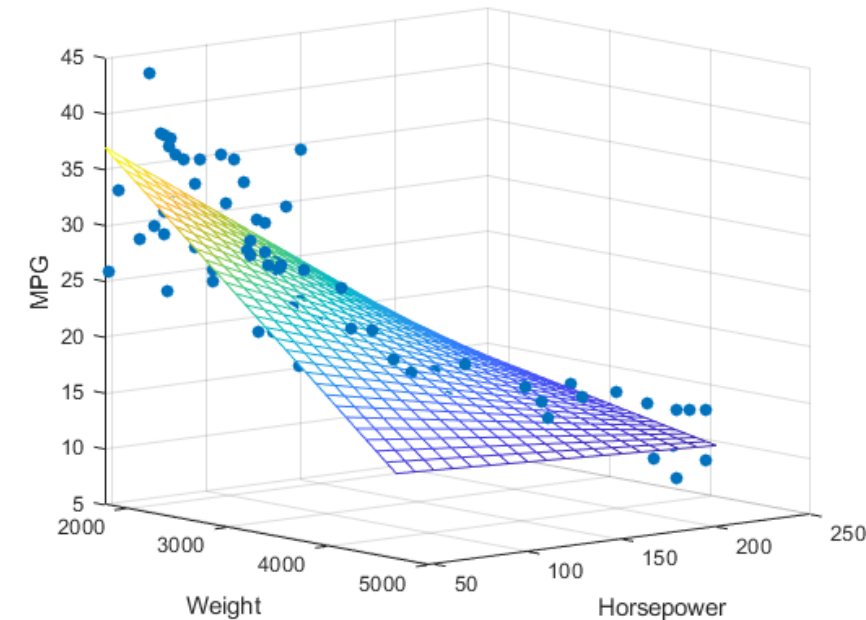


CONSTRUCCIÓN Y ENTRENAMIENTO DE MODELOS DE REGRESIÓN

Regresión lineal múltiple: Estimar variable numérica de salida y , a partir de n variables de entrada $x_1, x_2, \dots, x_n$, asumiendo una relación lineal:

$$\hat{y} = \theta_0 + \theta_1 x_1 + \dots + \theta_n x_n$$

- Modelo definido por $n+1$ parámetros o pesos θ_j
- El modelo pasa de ser una recta en 2D a ser un plano o, de forma más general, un hiperplano $(n+1)$ dimensional

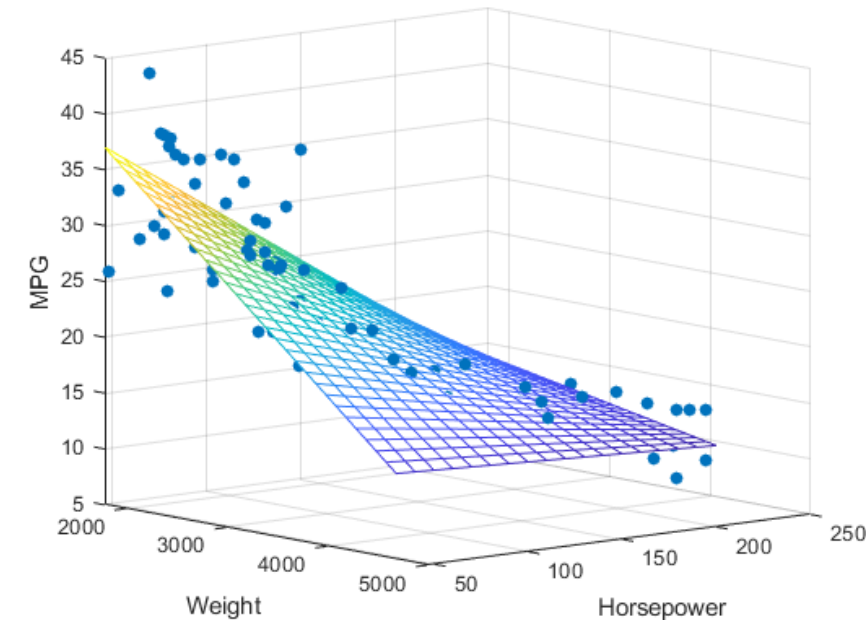


CONSTRUCCIÓN Y ENTRENAMIENTO DE MODELOS DE REGRESIÓN

Regresión lineal múltiple: Estimar variable numérica de salida y , a partir de n variables de entrada $x_1, x_2, \dots, x_n$, asumiendo una relación lineal:

$$\hat{y} = \theta_0 + \theta_1 x_1 + \dots + \theta_n x_n$$

- Modelo definido por $n+1$ parámetros o pesos θ_j
- El modelo pasa de ser una recta en 2D a ser un plano o, de forma más general, un hiperplano $(n+1)$ dimensional



<i>Tipo de parámetro</i>	<i>Notación</i>	<i>Descripción</i>
Pesos o importancia de los atributos (feature weights)	$\theta_1, \dots, \theta_n$	Un peso θ_j asociado a cada variable predictora. Un mayor peso indica que esa variable es más influyente en las predicciones del modelo.
Término de sesgo (bias term)	θ_0	Necesario para ajustarse a datos de entrenamiento que no pasen por el origen $(0, \dots, 0)$.

EL PROCESO INTERNO DE ENTRENAMIENTO DE UN MODELO

Objetivo: minimizar error en predicciones, definido por **función de pérdida (loss function)** $J(\theta)$:

- Diferencia entre valor real y , y predicción, $\hat{y} = \hat{y}(\theta)$.
- Se realiza el promedio sobre m predicciones de test.

MAE: *Mean Absolute Error* - usar cuando los datos contengan muchos *outliers* →

$$MAE = \frac{1}{m} |\hat{y}^{(i)} - y^{(i)}|$$

RMSE: *Root Mean Square Error* - más “pesimista” pero eficaz penalizando malas predicciones →

$$RMSE = \sqrt{\frac{1}{m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)})^2}$$

R² (Coeficiente de determinación): muy interpretable para evaluar el rendimiento, pues es una métrica que buscamos maximizar, a diferencia del error.

- Usar cuando necesitemos interpretabilidad

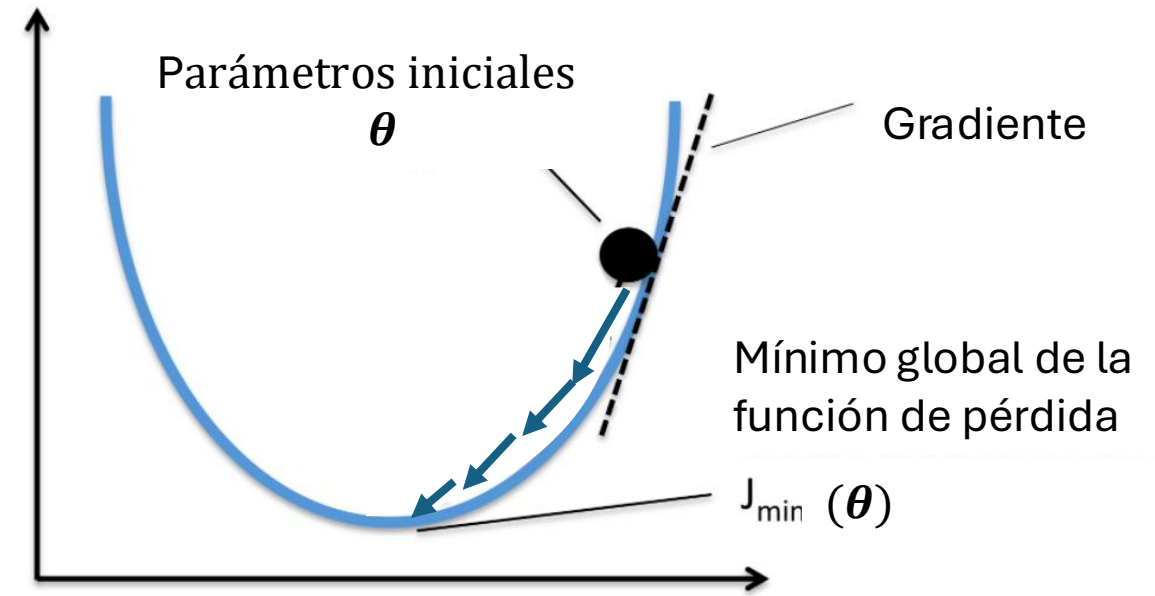
$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

R²: Coefficient of Determination
Σ: Summation
y_i: Actual Values
ŷ_i: Predicted Values
ȳ: Mean of Actual Values

EL PROCESO INTERNO DE ENTRENAMIENTO DE UN MODELO

Optimización por descenso de Gradiente

- Encontrar un conjunto de parámetros θ que minimicen la pérdida $J(\theta)$:
- Mediante cálculo de derivadas, acercarse todo lo posible al mínimo de la función de pérdida



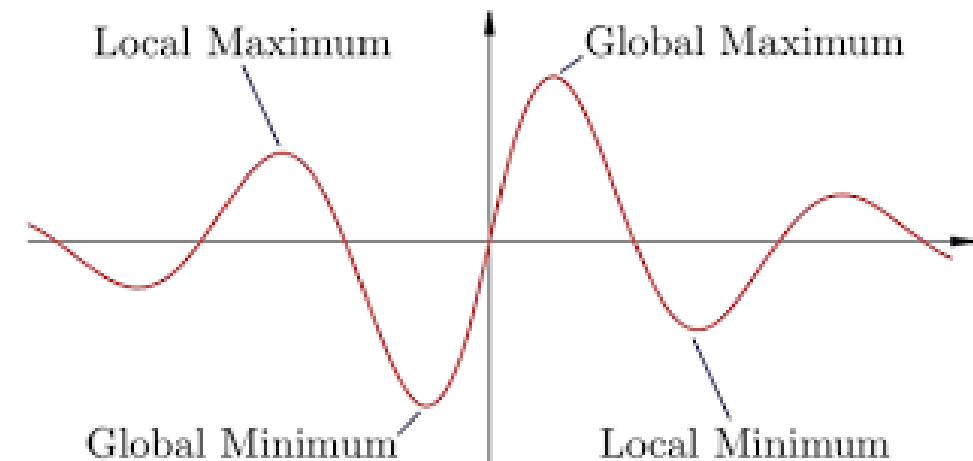
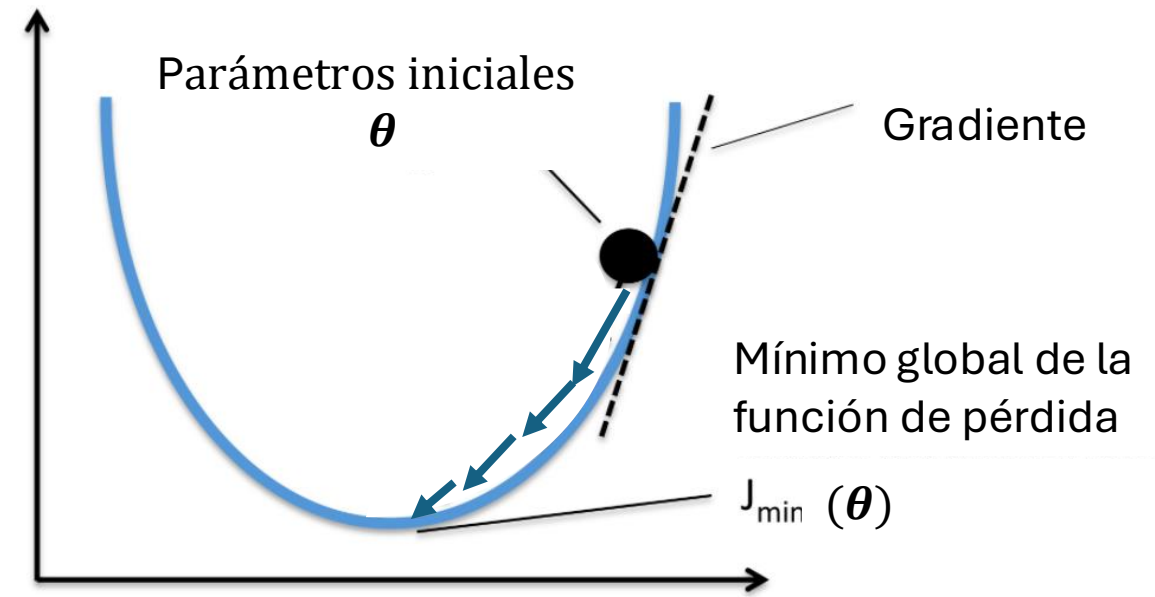
EL PROCESO INTERNO DE ENTRENAMIENTO DE UN MODELO

Optimización por descenso de Gradiente

- Encontrar un conjunto de parámetros θ que minimicen la pérdida $J(\theta)$:
- Mediante cálculo de derivadas, acercarse todo lo posible al mínimo de la función de pérdida

Stochastic Gradient Descent (SGD)

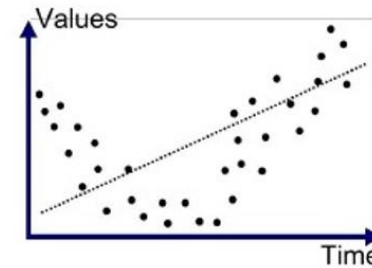
Añade aleatoriedad al proceso de optimización, haciéndolo más eficaz para grandes datasets complejos y funciones de pérdida donde podría haber mínimos locales.



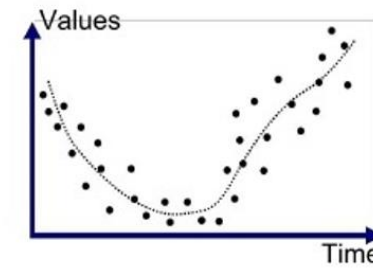
EL PROBLEMA DEL SOBREAJUSTE (OVERFITTING)

Aprender en exceso de los datos de entrenamiento podría no ser precisamente algo bueno

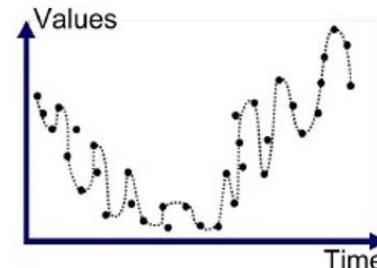
- “Memorizar” en lugar de aprender patrones generalizables a futuros ejemplos de datos no vistos durante el entrenamiento → sobreajuste
- Un buen modelo es el que no solo aprende de los datos de entrenamiento en sí, sino que es capaz de generalizar a futuros datos



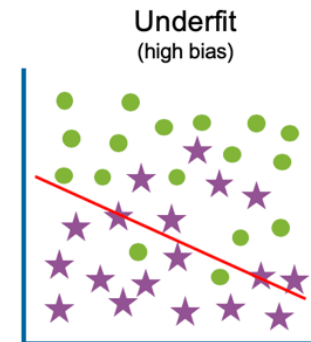
Underfitted



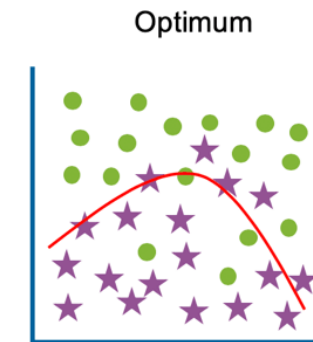
Good Fit/Robust



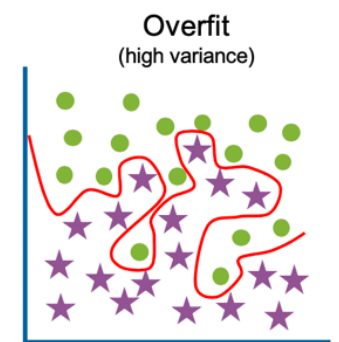
Overfitted



High training error
High test error



Low training error
Low test error

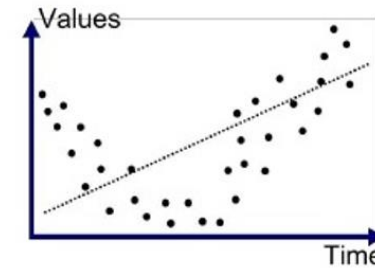


Low training error
High test error

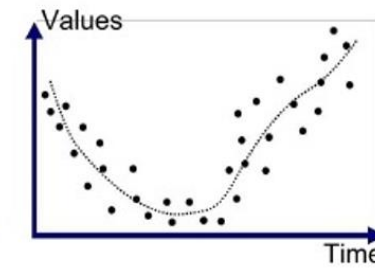
EL PROBLEMA DEL SOBREAJUSTE (OVERFITTING)

¿Cómo combatir el sobreajuste?

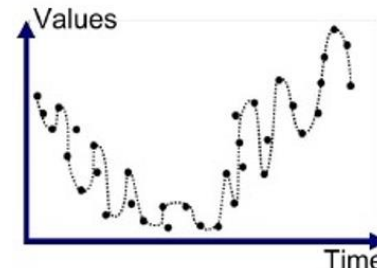
- **Simplificar el modelo:** elegir un tipo de modelo con menos parámetros, reducir atributos en el *dataset*, o restringe su capacidad de aprendizaje mediante técnicas de regularización.
- Recolectar más datos de entrenamiento
- Reducir el ruido en los datos de entrenamiento disponibles: errores e inconsistencias, *outliers*.



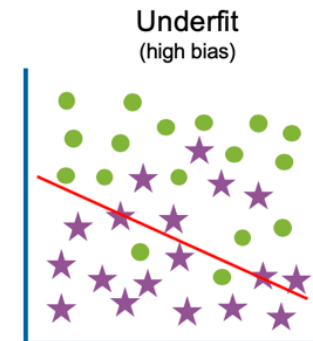
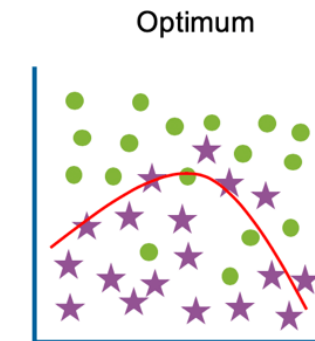
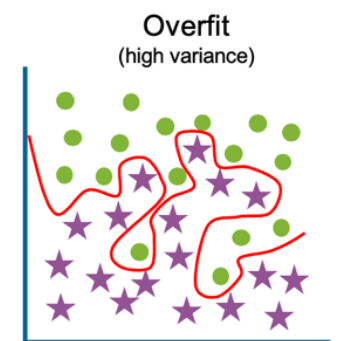
Underfitted



Good Fit/Robust



Overfitted

High training error
High test errorLow training error
Low test errorLow training error
High test error

EL DILEMA SESGO-VARIANZA (BIAS-VARIANCE TRADEOFF)

El error cometido por un modelo de ML a la hora de generalizar su capacidad predictiva a futuros datos puede deberse a varios factores, entre los que destacan dos: sesgo y varianza.

¿Qué es el sesgo (bias) de un modelo de ML?

- Un factor conducente a errores, debido a que el modelo ha realizado asunciones incorrectas sobre los datos, a raíz de los datos de entrenamiento.
- Por ejemplo, asumir que los datos son de naturaleza lineal cuando en realidad se asemejan más a una naturaleza cuadrática.
- Un modelo con alto sesgo es propenso a un mal ajuste a los datos de entrenamiento (underfitting).

EL DILEMA SESGO-VARIANZA (BIAS-VARIANCE TRADEOFF)

El error cometido por un modelo de ML a la hora de generalizar su capacidad predictiva a futuros datos puede deberse a varios factores, entre los que destacan dos: sesgo y varianza.

¿Qué es la varianza de un modelo de ML?

- Un factor conducente a errores, ya que el modelo entrenado es muy sensible a pequeñas variaciones en los datos de entrenamiento.
- Habitual en modelos con más parámetros o grados de libertad, p.ej. regresión polinomial de alto grado (los veremos en breve).
- Efecto común en modelos de alta varianza → sobreajuste (overfitting).

EN RESUMEN...

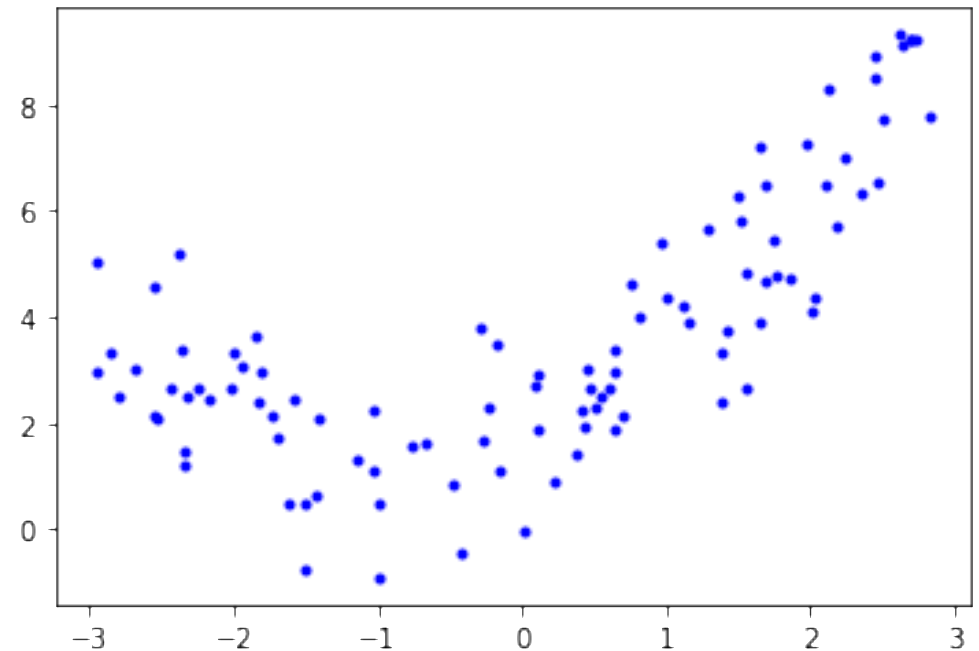
- (1) Aumentar la complejidad de un modelo → Mayor varianza y menor sesgo → Riesgo de *overfitting*
- (2) Entrenar un modelo muy simple → Mayor sesgo y menor varianza → Riesgo de *underfitting*

CONSTRUCCIÓN Y ENTRENAMIENTO DE MODELOS DE REGRESIÓN

Regresión polinomial: ¿Qué sucede cuando nuestros datos no se ajustan bien a una función lineal?

- Se puede “simular” una regresión lineal donde se creen nuevas variables que sean potencias (cuadrado, cubo, etc.) de las originales, e incluso productos entre éstas.

$$x^2_1, \dots, x^2_n, x^3_1, \dots, x^3_n, \dots, \\ x_1x_2, x_1x_3, \dots$$



CONSTRUCCIÓN Y ENTRENAMIENTO DE MODELOS DE REGRESIÓN

Regresión Lasso y Ridge: Son extensiones “naturales” de los modelos de regresión lineal, que incorporan la idea fundamental de la regularización para combatir el sobreajuste:

- Los **modelos más tradicionales** se entrenan guiados por el objetivo de **minimizar únicamente el error** (normalmente cuadrático)
- **En modelos polinomiales o lineales con muchos atributos**, esto suele aumentar el **riesgo de sobreajuste**.
- **Ridge (L2) y Lasso (L1)** son dos variantes que incorporan una función de penalización sobre los pesos que se van aprendiendo, para controlar su magnitud.

$$\min_{\beta} \|y - X\beta\|^2 + \alpha \|\beta\|_2^2$$

RIDGE (L2)

Penaliza pesos grandes de forma más suave que L1, sin llegar a reducirlos a 0

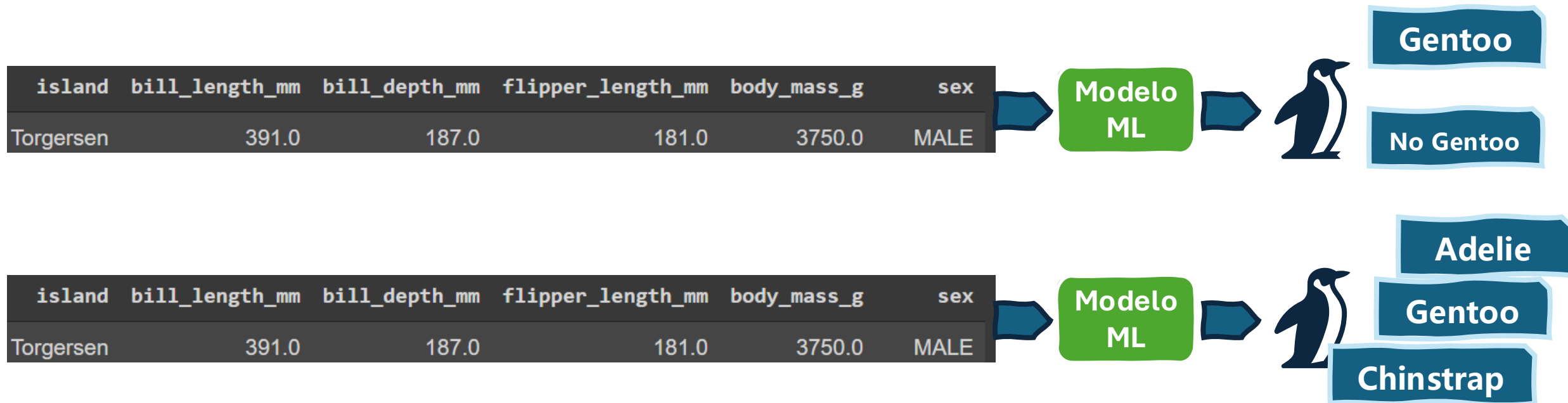
$$\min_{\beta} \|y - X\beta\|^2 + \alpha \|\beta\|_1$$

LASSO (L1)

Más agresivo, puede llegar a reducir algunos pesos a 0

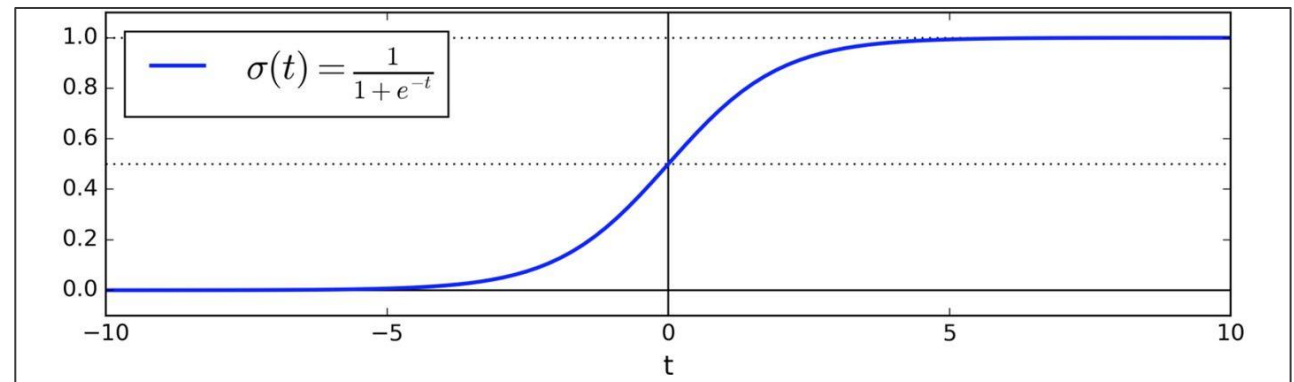
Modelos de clasificación

- Predecir la categoría o clase a la que pertenece una observación, de entre K clases posibles
- Clasificación binaria ($K = 2$) vs multi-clase ($K > 2$).



Regresión logística

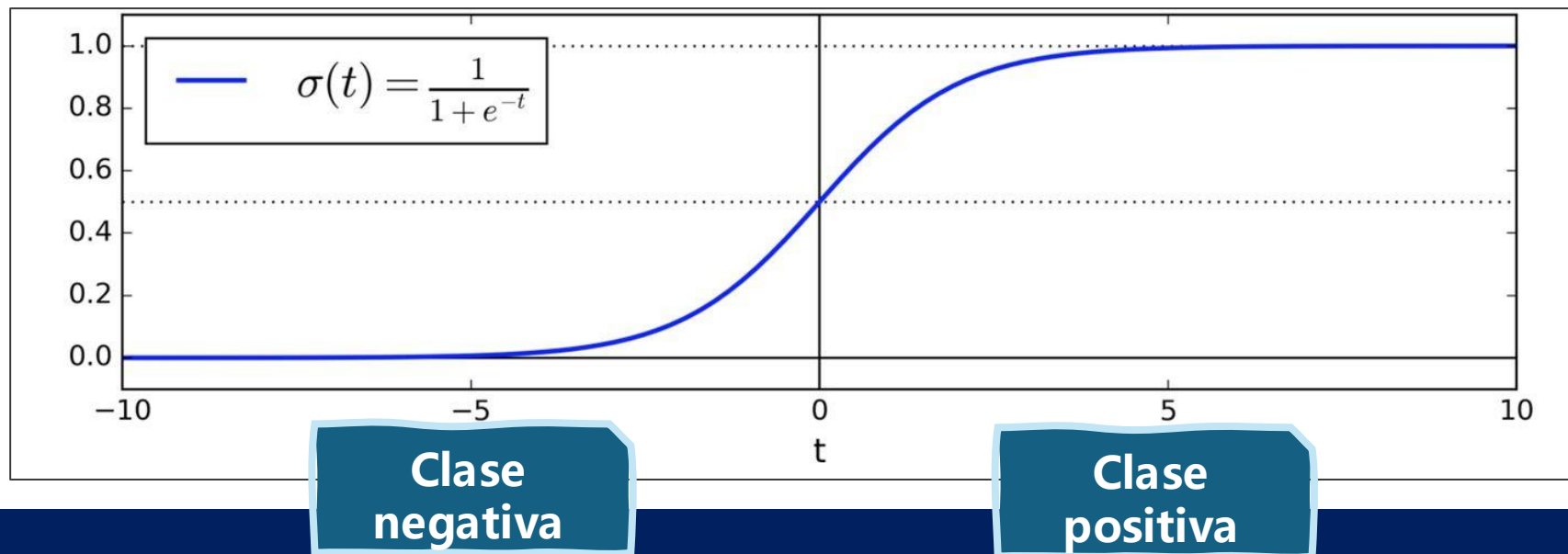
- Misma base matemática que la regresión lineal, pero reformulada para realizar tareas de **clasificación binaria**
- Dos clases mutuamente excluyentes
 - Positiva (1)
 - Negativa (0)
- Estimar probabilidad p de que una observación $\mathbf{x}$ pertenezca a la clase “positiva”.
 - $p \geq 0.5 \rightarrow$ predecir como “Si”
 - $p < 0.5 \rightarrow$ predecir como “No”



Regresión logística

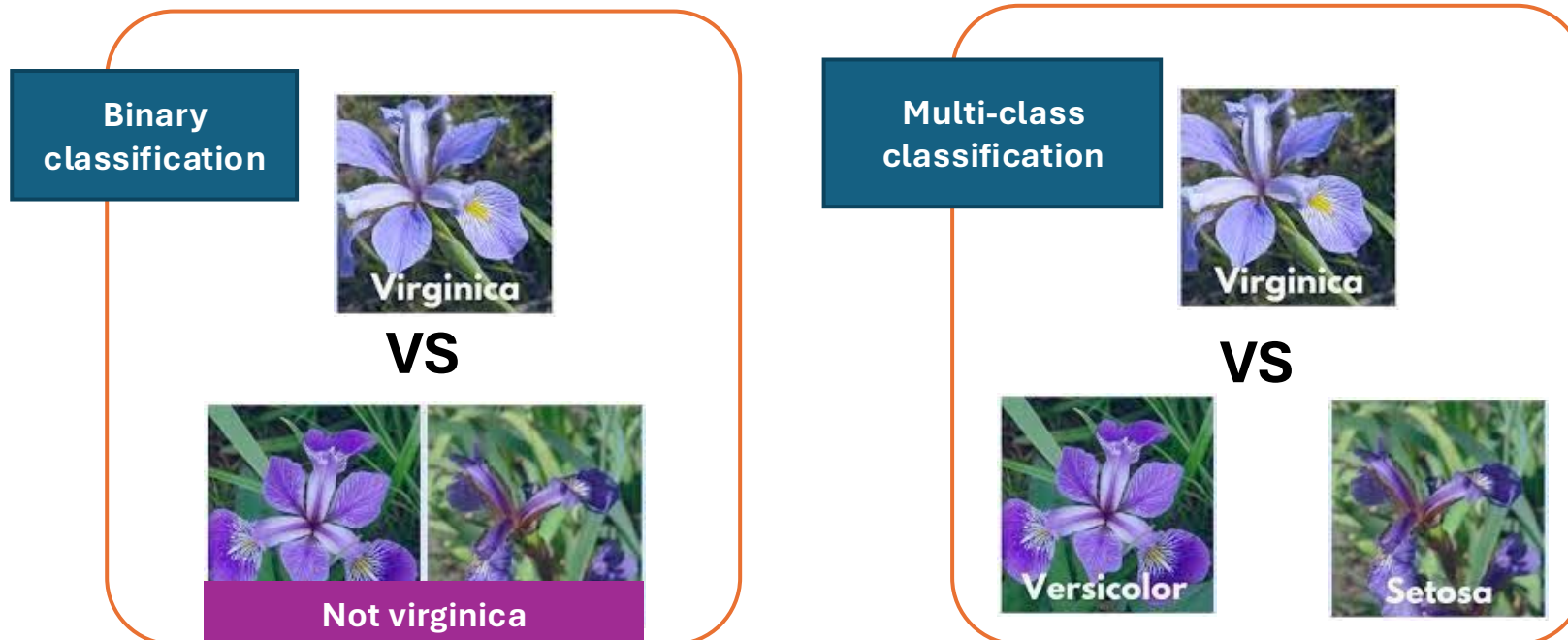
Predicción

- **Paso 1:** aplicar suma ponderada $\theta_0 + \theta_1 x_1 + \dots + \theta_n x_n$
- **Paso 2:** aplicar función logística para convertir resultado en una probabilidad de clase $\hat{p} = \sigma(\theta_0 + \theta_1 x_1 + \dots + \theta_n x_n)$



Regresión softmax

Contraparte de la regresión logística para acomodar múltiples (K) clases



Regresión softmax

Contraparte de la regresión logística para acomodar múltiples (K) clases

- **Paso 1:** calcular un score para cada clase k existente ($k = 1, \dots, K$)
 - $s_k(\mathbf{x}) = \theta_{k,0} + \theta_{k,1}x_1 + \dots + \theta_{k,n}x_n$
- **Paso 2:** aplicar función softmax, para obtener la probabilidad relativa de que $\mathbf{x}$ pertenezca a la clase k . Clasificar $\mathbf{x}$ con la clase de mayor probabilidad

$$\hat{p}_k = \sigma(s_k(\mathbf{x})) = \frac{\exp(s_k(\mathbf{x}))}{\sum_{j=1}^K \exp(s_j(\mathbf{x}))}$$

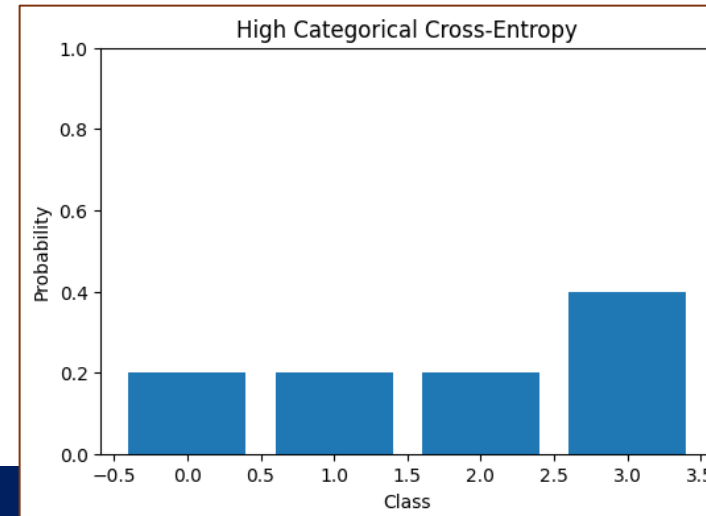
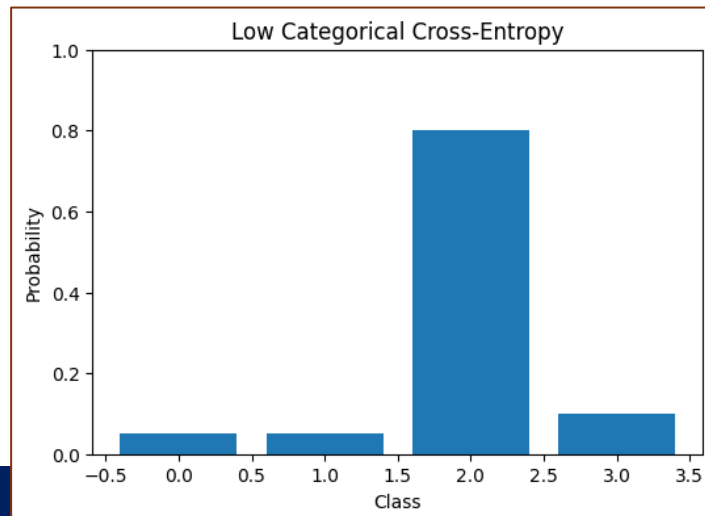
Número de pesos a aprender: $K \times (n+1)$

Regresión softmax

Entropía cruzada: métrica habitual para evaluar un clasificador durante su entrenamiento → función de costo

$$\text{Cross-entropy} = -\frac{1}{m} \sum_{i=1}^m \sum_{k=1}^K y_k^{(i)} \log(\hat{p}_k^{(i)})$$

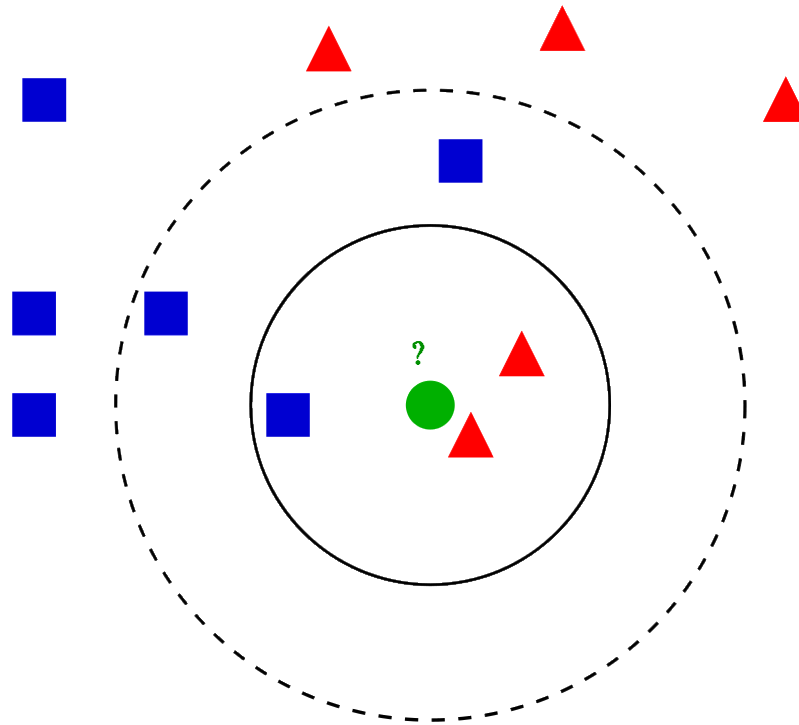
$\hat{p}_k^{(i)} = 1$ si la observación $\mathbf{x}_i$ pertenece realmente a la clase k , y $\hat{p}_k^{(i)} = 0$ en otro caso.



Clasificación de vecino más cercano (KNN)

Basado en instancias → no se construye un modelo

Se realiza la predicción observando directamente el conjunto de entrenamiento



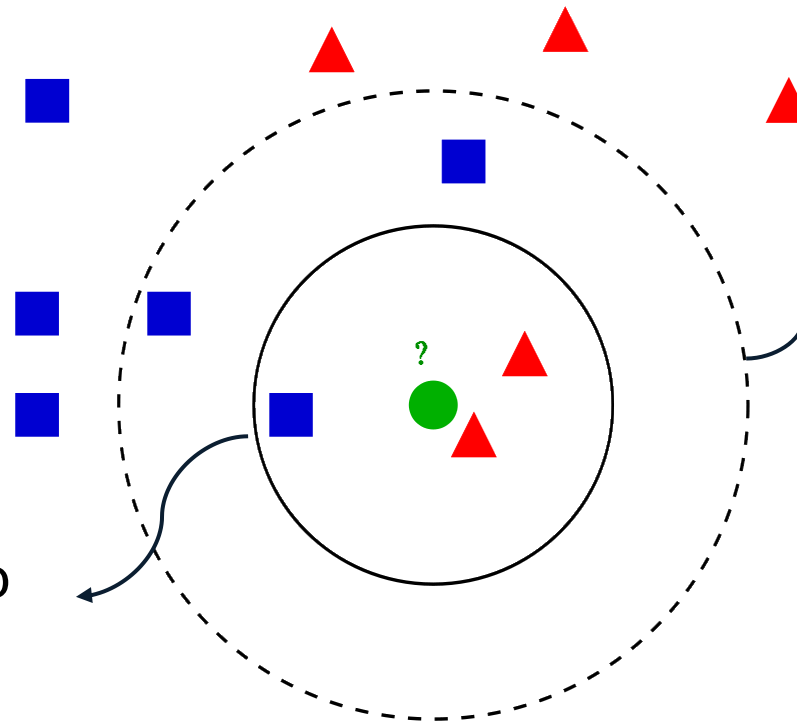
Clasificación de vecino más cercano (KNN)

Basado en instancias → no se construye un modelo

Se realiza la predicción observando directamente el conjunto de entrenamiento

Clasificar una nueva observación con la **clase mayoritaria de las K observaciones etiquetadas más similares**

k=3: clasificar como **triángulo**



K=5: clasificar como **cuadrado**

Clasificación de vecino más cercano (KNN)

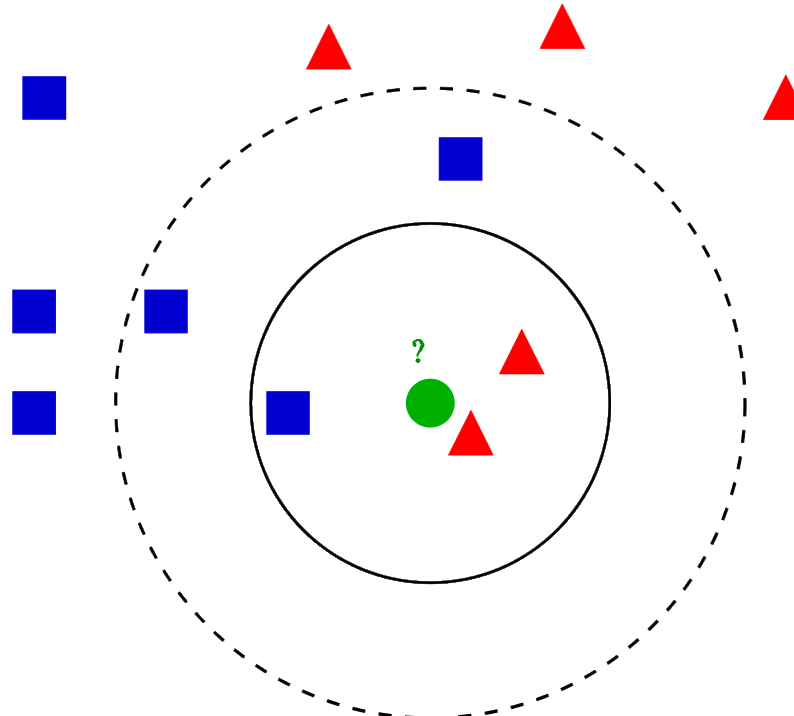
$$d(x, y) = \sum_{i=1}^n |x_i - y_i|$$

Distancia de Manhattan

$$d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2} = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2 + \dots + (x_n - y_n)^2}$$

Distancia Euclídea

Clasificar una nueva observación con la **clase mayoritaria de las K observaciones etiquetadas más similares**



Realizar una predicción requiere calcular las **similitudes entre la nueva observación y las existentes**:

$$\text{sim}(\mathbf{x}_t, \mathbf{x}_j) = 1 - d(\mathbf{x}_t, \mathbf{x}_j), \quad j = 1, \dots, m.$$

Clasificación de vecino más cercano (KNN)

¿Cómo elegir un buen K ?

Dependerá del problema y datos concretos: encontrar un k que produzca un alto nivel de acierto sobre un conjunto de test.

- Demasiado pequeño → **sobreajuste (overfitting)**. Caso extremo, $k=1$.
- Demasiado grande → **underfitting**, caso extremo $k=m$ (m = tamaño del conjunto de entrenamiento).
- Utilizar técnicas de **búsqueda de hiperparámetros**, que automáticamente encuentran una buena configuración.

Hiperparámetros

- Elegidos manualmente (o mediante búsqueda) a priori, consisten en parámetros que el algoritmo o modelo requiere configurar antes de entrenarlo, p.ej. k en un clasificador k -NN.

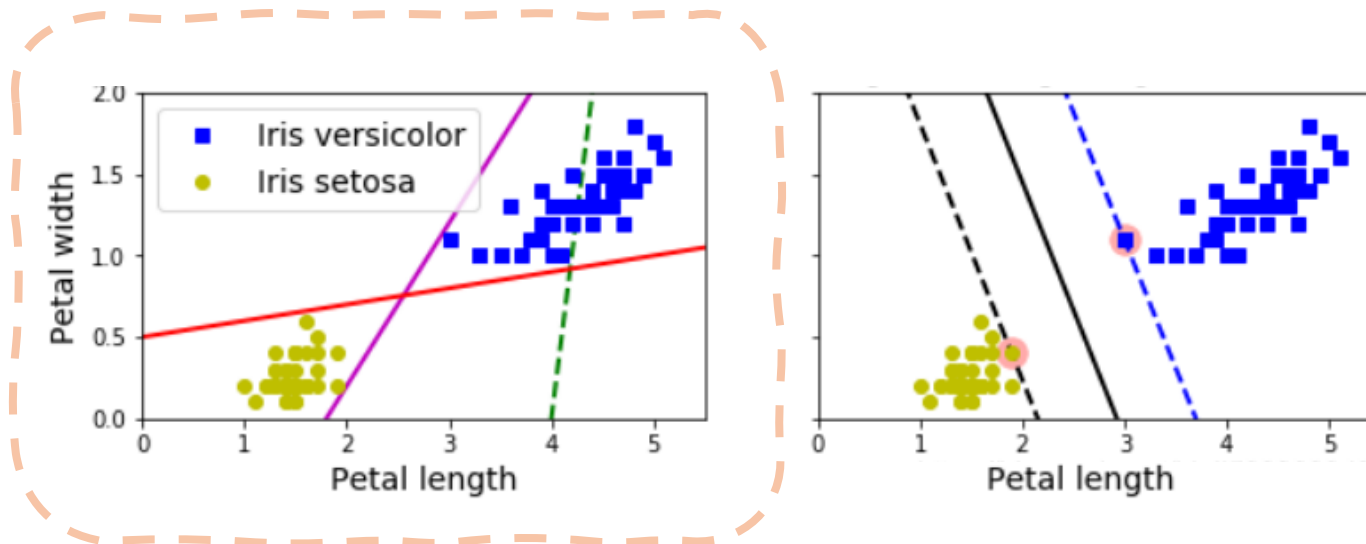
Parámetros

- En modelos de ML, son los “pesos” que no hace falta especificar a priori, ya que el modelo los aprenderá a partir de un *dataset* durante el entrenamiento, p.ej. coeficientes en un modelo de regresión lineal.

Máquinas de vectores soporte (SVM)

Modelos versátiles capaces de realizar...

- Clasificación lineal o no lineal → datos complejos de tamaño bajo-medio
- Regresión

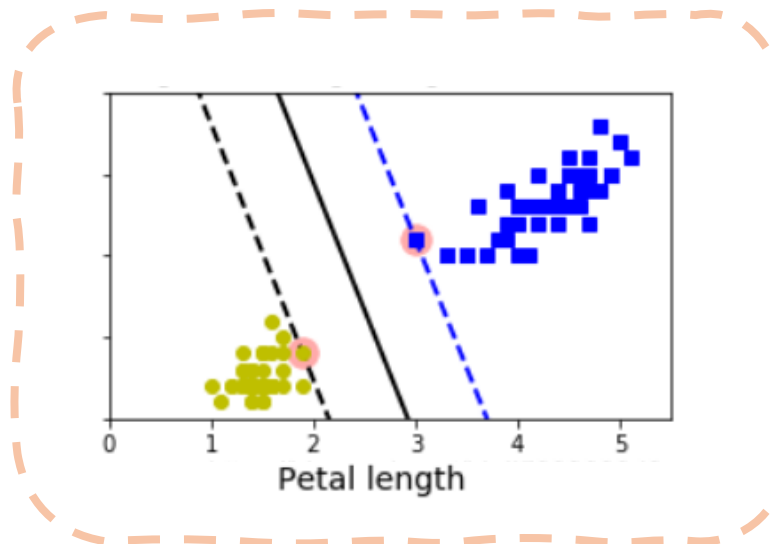
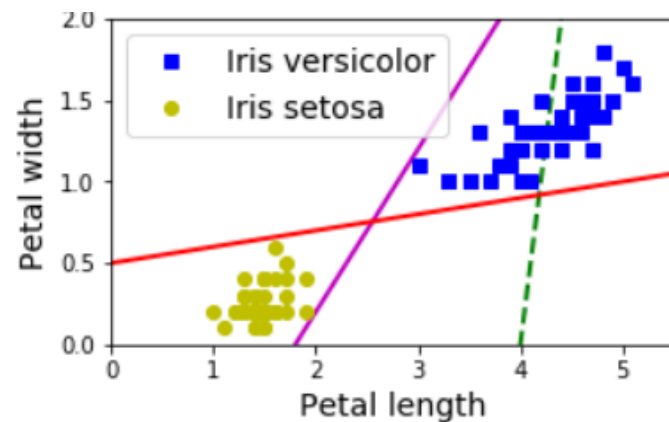


En un clasificador sencillo como regresión logística, las fronteras de decisión entre clases predichas pueden realizar una separación muy eficaz, pero algunos puntos quedan demasiado cerca de la frontera.

Máquinas de vectores soporte (SVM)

Modelos versátiles capaces de realizar...

- Clasificación lineal o no lineal → datos complejos de tamaño bajo-medio
- Regresión

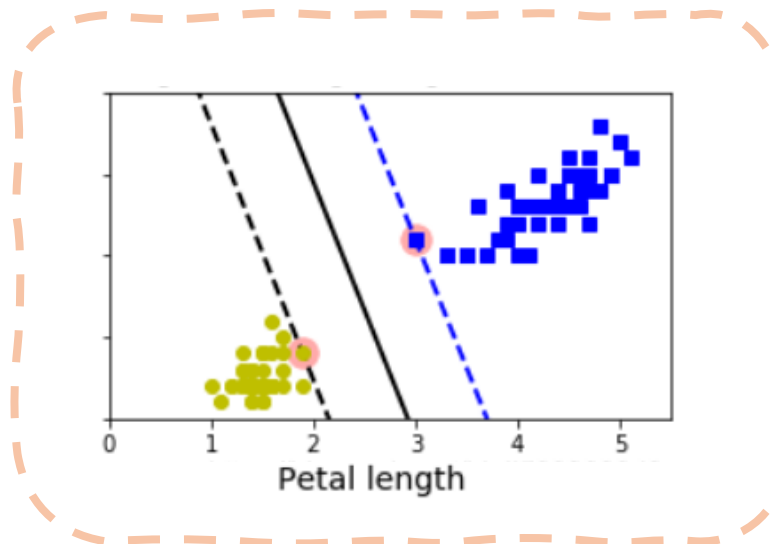
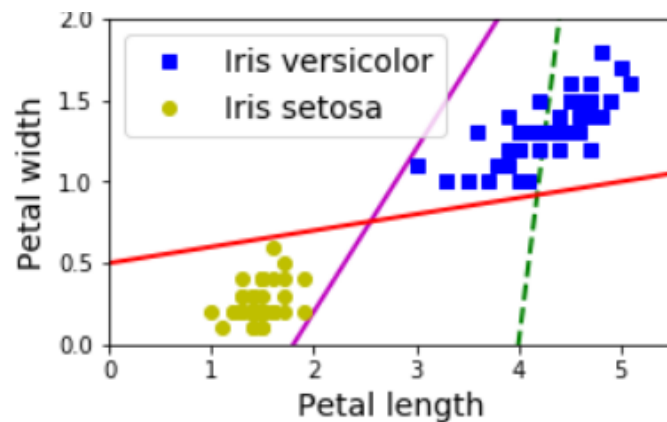


Un clasificador SVM no solo procura separar bien las clases sino también colocar la frontera de decisión lo más lejos posible de las **instancias de entrenamiento más cercanas: vectores soporte**

Máquinas de vectores soporte (SVM)

Modelos versátiles capaces de realizar...

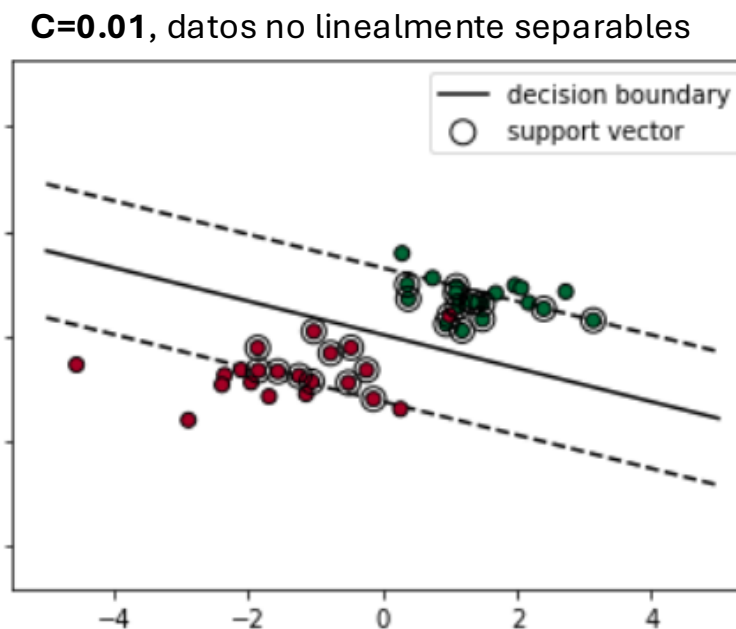
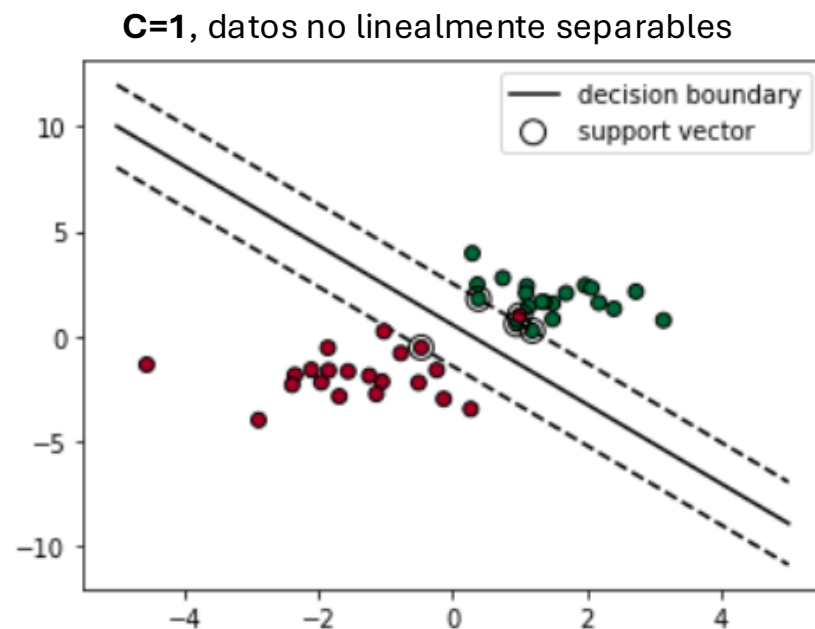
- Clasificación lineal o no lineal → datos complejos de tamaño bajo-medio
- Regresión



SVM es muy sensible a atributos con escalas muy diferentes, p.ej. Edad e ingresos anuales. Conviene escalar los atributos *antes* de entrenar el modelo

Máquinas de vectores soporte (SVM)

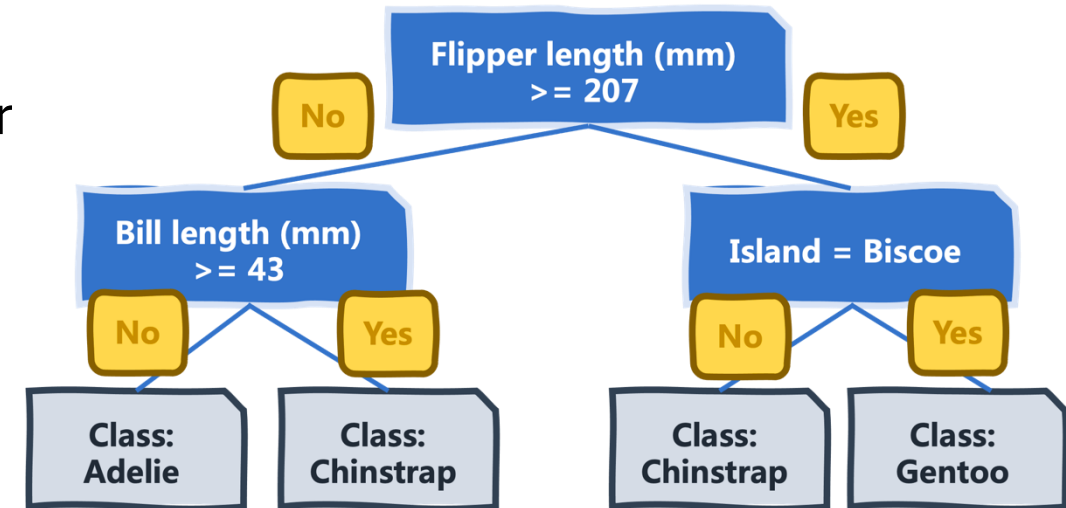
El **hiperparámetro c** se puede ajustar para controlar cómo de estricto debe ser el modelo con este criterio, pudiendo a veces dejar puntos dentro de los márgenes laterales. Un **c** demasiado alto, con márgenes estrechos, puede suponer mayor riesgo de *overfitting*



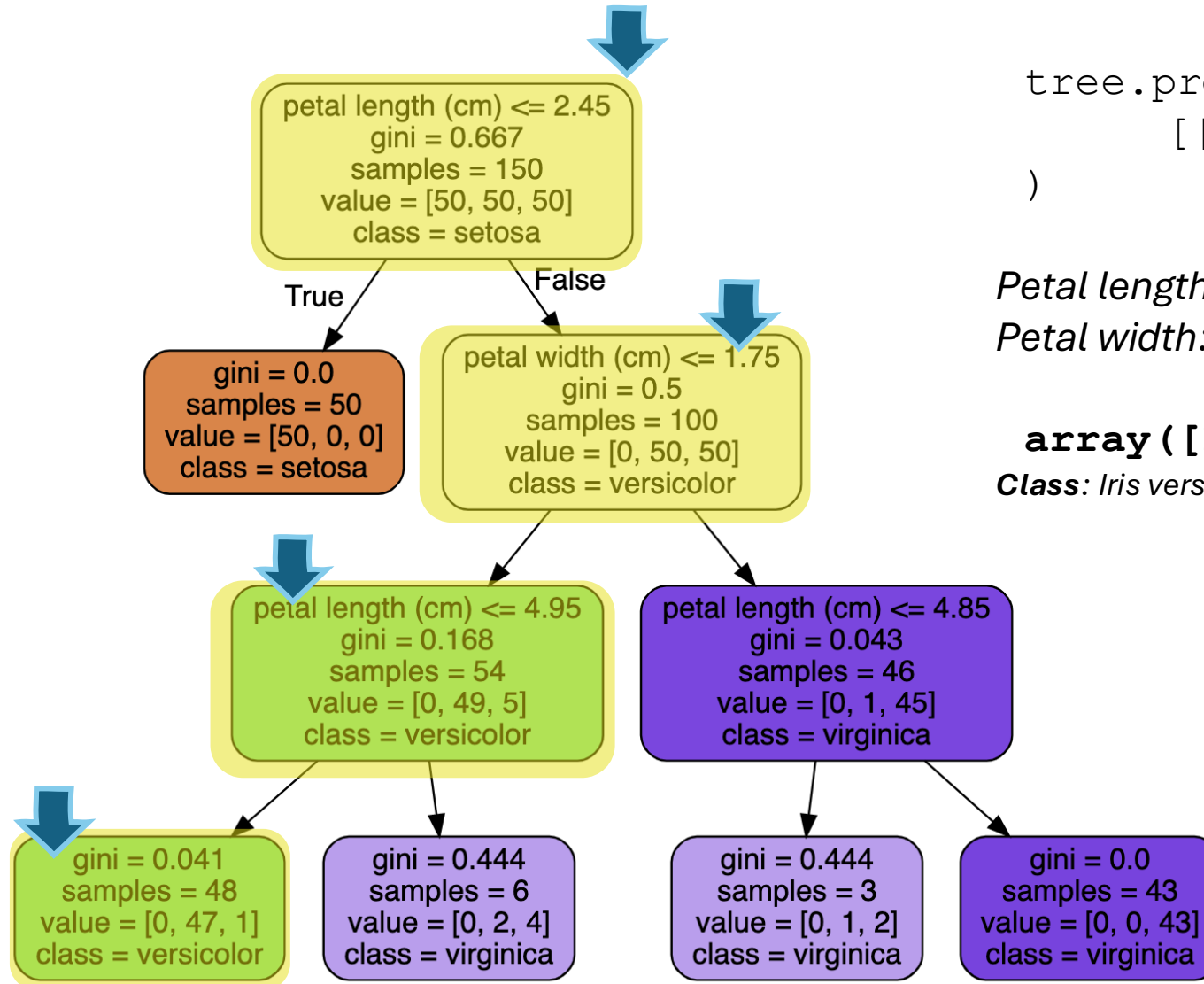
Árboles de decisión

Modelos intuitivos para clasificación y regresión

- Más efectivos que otros modelos para clasificar datos de complejidad media
- Fácil de interpretación de las predicciones: modelos de **caja blanca**
- Elemento básico de los bosques aleatorios (*random forests*)
- No paramétrico → el modelo no viene descrito por un conjunto de parámetros o pesos



Árboles de decisión



```
tree.predict(  
    [[2.5, 1.7]]  
)
```

Petal length: 2.5
Petal width: 1.7

array([1])
Class: *Iris versicolor*

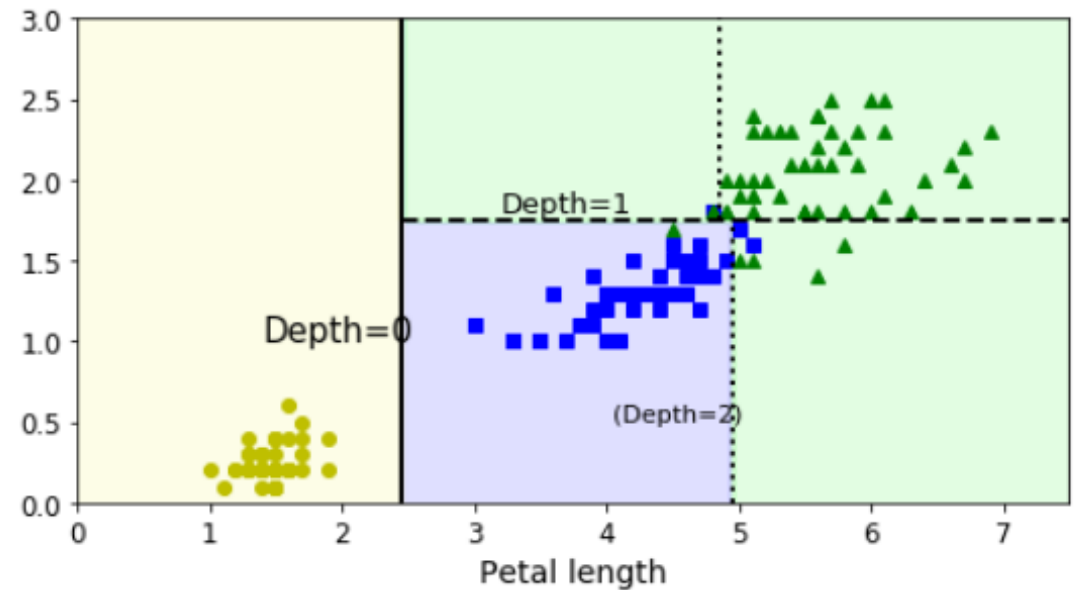
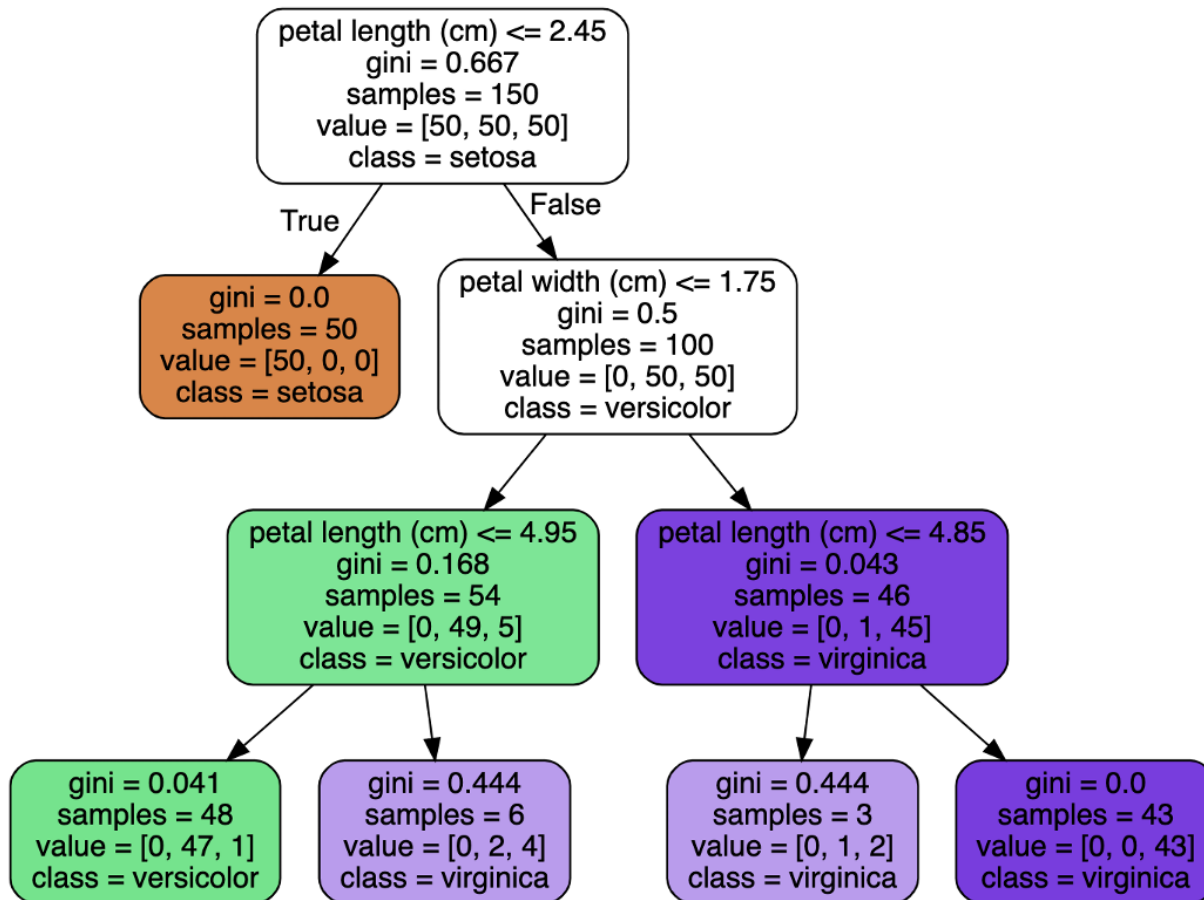


Clasificar ejemplares de flores de la familia iris (tres clases):

0: setosa
1: versicolor
2: virginica

Árboles de decisión

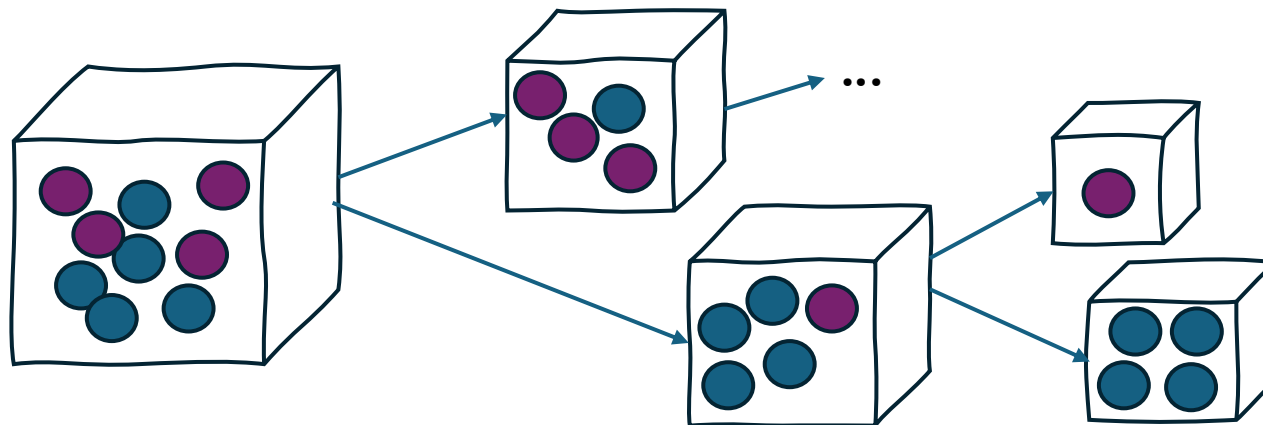
Fronteras de decisión del modelo: una para cada partición realizada de los datos de entrenamiento



Árboles de decisión

Cómo se construye el árbol

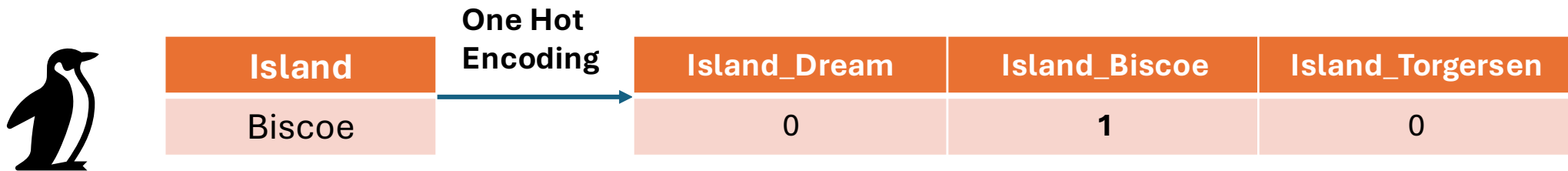
- Particionar gradualmente el conjunto de entrenamiento según clases, buscando **maximizar la homogeneidad de clase** en las particiones hechas.
- Criterio de partición: un valor umbral de uno de los atributos del dataset
- Métricas de impureza: entropía, índice Gini.
 - Se busca minimizar la impureza.



Árboles de decisión

Uso de atributos categóricos en árboles de decisión

- Si hay un orden lógico (p.ej. Talla de camiseta 'S', 'M', 'L', 'XL') codificar con valor entero que respete el orden.
- Si no hay orden lógico, aplicar codificación: **OneHotEncoding**.



Evaluación de clasificadores

Matriz de confusión

- Muestra predicciones correctas e incorrectas por clase.
- Diagonal principal = predicciones acertadas

		Predicted Label	
		Positive	Negative
Actual Label	Positive	TRUE POSITIVE TP	FALSE NEGATIVE FN
	Negative	FALSE POSITIVE FP	TRUE NEGATIVE TN

		Predicted class			Actual Total
		Iris-setosa	Iris-versicolor	Iris-virginica	
Actual class	Iris-setosa	19	0	0	19
	Iris-versicolor	0	15	0	15
	Iris-virginica	0	1	15	16
	Predicted Total	19	16	15	50

Evaluación de clasificadores

Precision (término inglés, no "precisión")

- % de predicciones de una clase hechas correctamente

$$precision = \frac{TP}{TP + FP}$$

Recall (sensibilidad)

- % instancias cuya clase real ha sido correctamente predicha

$$recall = \frac{TP}{TP + FN}$$

F1 Score

- Media armónica de precision y recall. Busca un equilibrio entre ambas

$$F1 = 2 \frac{precision \times recall}{precision + recall}$$

		Predicted Label	
		Positive	Negative
Actual Label	Positive	TRUE POSITIVE TP	FALSE NEGATIVE FN
	Negative	FALSE POSITIVE FP	TRUE NEGATIVE TN

Evaluación de clasificadores

Curva ROC y Área bajo la curva (AUC)

- Curva que representa el TPR (ratio de verdaderos positivos) en función del FPR.
- Un clasificador es tanto “mejor” cuanto más se acerca su curva ROC al punto (0,1).
 - **Mayor área bajo la curva**

